

Mensajes de error: diseño seguro y observable

Riesgo técnico

Exponer trazas (stack traces), nombres internos de clases/tablas, consultas SQL u otros detalles de infraestructura en respuestas HTTP permite a un atacante perfilar el sistema y preparar exploits (SQLi, RCE, traversal, evasión de WAF).

Principios

- **Separación de audiencias:**
 - *Cliente* → mensaje **genérico** y código HTTP correcto.
 - *Operación/Seguridad* → detalle completo en **logs**/trazas internas.
- **Consistencia:** esquema de error estable y versionado (evita “parseo” frágil por parte del cliente).
- **No filtrar secretos:** nunca incluir tokens, llaves, rutas sensibles, IPs privadas o stack traces en la respuesta.

Esquema recomendado (ejemplo)

```
{  
  "error": {  
    "code": "RESOURCE_NOT_FOUND",  
    "message": "El recurso no existe.",  
    "correlation_id": "2b8f1c7e-..."  
  }  
}
```

- **code** → identificador estable para reglas de cliente/monitorización.
- **message** → legible, sin revelar detalles internos.
- **correlation_id** → une la respuesta con logs y trazas.

Buenas prácticas

- Modo **producción**: mensajes genéricos, sin `stacktrace`.
- Modo **desarrollo**: permitir detalle solo bajo `DEBUG=true` y solo en entornos no públicos.
- **Mapeo exhaustivo** a HTTP: 400/401/403/404/409/422/429/5xx; evita devolver 200 con errores en el cuerpo.
- **Rate limit y circuit breakers**: impiden extracción masiva de diferencias de error como canal lateral.
- **Observabilidad**: registra el detalle técnico en logs/trace con `correlation_id`.

Auditoría y revisión de logs: del evento crudo al hallazgo

Riesgo técnico

Sin registro ni revisión de logs, no hay trazabilidad ni detección temprana de abuso (fuerza bruta, scraping, exploración de endpoints).

Objetivo

Construir una **línea de auditoría** confiable (quién, qué, cuándo, desde dónde) y explotable (búsqueda, agregaciones, alertas).

Contenido mínimo por evento

- **Contexto**: `timestamp`, `service`, `environment`, `version`, `host/ip`.
- **Request**: `method`, `path`, `query_hash` (no almacenar query en claro si hay PII), `headers_seleccionados` (nunca tokens), `client_ip`.
- **Respuesta**: `status`, `latency_ms`, `size`.
- **Identidad**: `user_id/subject` si aplica.
- **Correlación**: `correlation_id/trace_id`.
- **Categoría**: `auth`, `access_denied`, `validation_error`, `server_error`, etc.

Procesamiento

- **Normalización** (Logstash/Fluentd): asegurar tipos (números para latencias; **keyword** para IDs), anonimizar PII (hash/suppress).
- **Almacenamiento** (Elasticsearch/ClickHouse): índices rotados por fecha y entorno; políticas de retención (ILM).
- **Explotación** (Kibana/Grafana/Loki): búsquedas guardadas (“5xx últimos 15m”, “explosión de 401”), dashboards por servicio.
- **Alertas**: umbrales/ratios (picos de 401/403, tasa 5xx, latencia p95 disparada, errores repetitivos de validación).

Controles operativos

- **Integridad** del pipeline (reintentos, colas, backpressure).
- **Gobernanza** de acceso a logs (principio de menor privilegio).
- **Trazabilidad legal** (retención alineada a normativa; redacción de PII).

Autenticación por tokens: modelos, propiedades y validaciones

Riesgo técnico

Sin autenticación robusta, cualquier actor puede acceder a recursos sensibles. Autenticaciones débiles (tokens sin firma/verificación, TTL excesivo) facilitan secuestro de sesión.

Modelos comunes

- **API Keys opacas**: string aleatorio almacenado/validado server-side. Simples; rotación y escopes manuales.
- **JWT (JSON Web Token)**: *self-contained* (claims firmadas).
 - **Firma**: RS256/ES256 recomendado (asimétrica); evita HS256 compartiendo secret en múltiples servicios si no es necesario.
 - **Claims críticos**: **iss**, **sub**, **aud**, **exp**, **iat**, opcional **nbf**.

- **Validaciones:**
 - Verificar **firma** y algoritmo.
 - Comprobar **exp/nbf/iat** con **tolerancia** de reloj (skew).
 - Restringir **aud** al servicio destinatario.
 - Comprobar **revocación** (lista o *token introspection* si es OAuth2).
- **Scopes/Roles:** autorización por **permisos**; no solo “está logueado”.
- **OAuth2/OIDC:** *authorization code* + **access token** (corto) y **refresh token** (seguro, rotado).
- **mTLS:** autenticación por certificado cliente (alto assurance, más operativo).

Propiedades deseables del token

- **TTL corto** (minutos) para access tokens; refresh rotado.
- **No reusable** tras logout/rotación (revocación efectiva).
- **Bound** al cliente (DPoP/MTLS) cuando el riesgo lo amerita.
- **Transporte** únicamente por **HTTPS**, sin almacenamiento en lugares inseguros (localStorage con XSS; preferir cookies **HttpOnly** + **SameSite** cuando aplique).

Errores frecuentes

- Aceptar cualquier **alg** o desactivar **verify_signature**.
- No validar **aud/iss**.
- TTLs de días/semanas.
- Incluir PII amplia en el JWT (recorre múltiples servicios y logs).

Diferenciar 401 vs 403: semántica, telemetría y DX

Riesgo técnico

Respuestas mal mapeadas ocultan la causa real: *no autenticado* vs *autenticado sin permiso*. Perjudica observabilidad, UX y capacidad de respuesta ante incidentes.

Definiciones

- **401 Unauthorized** (*en realidad “Not Authenticated”*):
 - El cliente **no se ha autenticado** o el token es inválido/expirado.
 - Debe acompañarse de **WWW-Authenticate** cuando procede (p. ej., **Bearer**).
- **403 Forbidden**:
 - El cliente **está autenticado**, pero **carece de permisos** para esa acción/objeto.

Reglas prácticas

- Validar **autenticación** antes que **autorización**.
- Si falla autenticación → **401** (mensaje genérico).
- Si pasa autenticación y falla autorización (RBAC/ABAC, *ownership*, políticas) → **403**.
- Registrar en logs:
 - 401 → motivo (“token_expired”, “token_invalid”, “missing_header”).
 - 403 → política/regla fallada (“role_missing:admin”, “not_owner”).
- Métricas separadas por **status** y **handler** para entender: picos de 401 (problemas de login/expiración) vs picos de 403 (mal *scoping* o abuso).

DX (Developer Experience) y seguridad

- Mensajes **claros pero no verbosos**: no confirmar existencia de recursos protegidos si el permiso es el problema (evitar oráculos).
- Documentar contratos de error en OpenAPI para clientes.

Monitoreo básico que sostiene los cuatro frentes

Qué medir

- **Disponibilidad**: tasa de 5xx global y por endpoint.
- **Autenticación/Autorización**: tasa separada de **401** y **403**; ratio 401/403.
- **Rendimiento**: latencia **p95** por endpoint; RPS.
- **Seguridad operativa**: contador de intentos de autenticación fallidos; frecuencia de errores de validación.

Cómo instrumentar

- **Métricas**: contadores por **status**, histogramas de duración etiquetados por **handler**.
- **Logs**: eventos estructurados con **correlation_id** y causa de 401/403.
- **Trazas**: spans con tags **http.status_code**, **endpoints**, errores; relacionar con logs.
- **Alertas**: umbrales en tasas (5xx, 401, 403) y latencia p95; *anomaly detection* si está disponible.

Operación continua

- Revisiones periódicas de patrones en logs (picos fuera de horario, IPs anómalas).
- Rotación/validación de tokens y claves.
- Pruebas de seguridad regulares (pasivas/activas) en staging y producción controlada.
- Postmortems con evidencias (métricas + logs + trazas) para acciones correctivas.

